

Arrays

```
int[] v = {1, 2, 3}; // Cria com valores
int[] v2 = new int[5]; // Vetor vazio tam 5
v[0] = 10; // Altera pos 0
int size = v.length; // Tamanho: 3
for (int x : v) {} // Loop (for-each)
```

ArrayList

```
import java.util.ArrayList;
ArrayList<String> l = new ArrayList<>();
l.add("A"); // Adiciona elemento
l.add(0, "B"); // Insere na pos 0
String val = l.get(1); // Retorna pos 1 -> "A"
l.set(1, "C"); // Altera pos 1
l.remove(0); // Remove pos 0
int sz = l.size(); // Tamanho da lista
```

Métodos

```
// Definição de método estático
public static int soma(int a, int b) {
    return a + b; // Retorno
}
// Chamada do método:
int res = soma(5, 3); // res = 8
```

Conversões

```
// String para Numérico e vice-versa
int i = Integer.parseInt("123"); // Str -> int
double d = Double.parseDouble("4.5"); // Str -> double
String s1 = String.valueOf(10); // int -> Str
String s2 = Integer.toString(20); // int -> Str
```

Collections.sort

```
import java.util.Collections;
import java.util.ArrayList;
ArrayList<String> l = new ArrayList<>();
l.add("B"); l.add("A");
Collections.sort(l); // Ordena: ["A", "B"]
Collections.reverse(l); // Inverte: ["B", "A"]
```

Overload (Sobrecarga)

```
// Mesmo nome, assinaturas diferentes
void print(int i) {}
void print(String s) {}
void print(int i, String s) {}
```

Setter

```
public class User {
    private String nome;
    public void setNome(String n) { // Setter
        this.nome = n;
    }
}
```

static

```
public class Contador {
    public static int total = 0; // Atributo de classe
    public static void inc() { total++; } // Método classe
}
// Uso (sem instanciar):
Contador.total = 5;
Contador.inc();
```

null

```
String s = null; // Sem referência
// Evita NullPointerException:
if (s != null && s.equals("ok")) {
    System.out.println("Ok");
}
```

continue

```
for (int i = 0; i < 5; i++) {
    if (i == 2) continue; // Pula iteração
    System.out.print(i); // Imprime 0134
}
```

Matrizes (Multidimensionais)

```
int[][] m = {{1, 2}, {3, 4}}; // Inicializa 2x2
int[][] m2 = new int[3][3]; // Vazia 3x3
m[0][1] = 5; // Altera L0, C1
int rows = m.length; // Linhas: 2
int cols = m[0].length; // Colunas L0: 2
```

String

```
String s = "Java"; // Cria literal
int len = s.length(); // Tamanho: 4
char c = s.charAt(0); // Char na pos 0: 'J'
String sub = s.substring(0,2); // Substring: "Ja"
boolean eq = s.equals("java"); // Compara valor: false
boolean eqI = s.equalsIgnoreCase("java"); // true
String low = s.toLowerCase(); // "java"
```

Math

```
double sq = Math.sqrt(9.0); // Raiz: 3.0
double p = Math.pow(2, 3); // Potência: 8.0
int val = Math.abs(-10); // Absoluto: 10
double r = Math.random(); // Rand em [0.0, 1.0)
long rd = Math.round(5.6); // Arredonda: 6
int mx = Math.max(10, 20); // Retorna maior: 20
```

Objetos

```
Pessoa p = new Pessoa("Ana"); // Instancia
p.falar(); // Executa método
p = null; // Remove referência
```

Arrays.sort

```
import java.util.Arrays;
int[] v = {3, 1, 2};
Arrays.sort(v); // Ordena: {1, 2, 3}
int idx = Arrays.binarySearch(v, 2); // Busca: 1
```

Cast (Coerção)

```
// Implícito (Sem perda)
int i = 10;
double d = i; // Auto (Widening)
// Explícito (Pode perder dados)
double x = 9.78;
int y = (int) x; // y = 9 (Narrowing)
```

Getter

```
public class User {
    private String nome;
    public String getNome() { // Getter
        return this.nome;
    }
}
```

Encapsulamento

```
public class Conta {
    private double saldo; // Atributo privado
    public double getSaldo() { return saldo; }
    public void depositar(double v) {
        if (v > 0) saldo += v; // Validação
    }
}
```

instanceof

```
Animal a = new Cachorro();
if (a instanceof Cachorro) { // true
    Cachorro c = (Cachorro) a; // Cast seguro
}
```

break

```
for (int i = 0; i < 10; i++) {
    if (i == 5) break; // Sai do loop
    System.out.print(i); // Imprime 01234
}
```

Classe

```
// Molde/Template do objeto
public class Carro {
    String marca;           // Estado
    void acelerar() {}      // Comportamento
}
```

Construtor

```
public class Item {
    String nome;
    // Executado no 'new'
    public Item(String nome) {
        this.nome = nome;
    }
}
```

Getter

```
public class Pessoa {
    private int idade;
    public int getIdade() { // Acesso ao atributo
        return this.idade;
    }
}
```

super

```
class Pai { Pai(int x){} }
class Filho extends Pai {
    Filho(int x) {
        super(x); // Construtor do Pai
    }
    void msg() {
        super.hashCode(); // Método do Pai
    }
}
```

Classe Abstrata

```
abstract class Figura {
    abstract void draw(); // Sem corpo (abstrato)
    void show() {}        // Concreto permitido
}
```

final

```
final class Constante {} // Sem subclasses
class Teste {
    final int VAL = 10;    // Constante
    final void exibe() {}  // Sem override
}
```

toString()

```
// Representação textual
@Override
public String toString() {
    return "User[nome=" + nome + "]";
}
```

Associação

```
// Relacionamento genérico
class Motorista {
    Carro carro; // "Tem um" (sem herança)
}
```

Modificadores de acesso

```
public class Acesso {
    public int a; // Qualquer classe acesse
    protected int b; // Subclasse ou mesmo pacote
    int c; // Mesmo pacote (default)
    private int d; // Apenas esta classe
}
```

Objeto

```
// Instanciação da classe
Carro c1 = new Carro(); // Cria c1
Carro c2 = new Carro(); // Cria c2 (indep)
```

Métodos

```
public class Calc {
    // Retorno e parâmetros
    public int somar(int x, int y) {
        return x + y;
    }
}
```

Encapsulamento

```
// Esconde atributos; expõe métodos
public class Banco {
    private double saldo; // Só a classe lê/escreve
    public double getSaldo() { return saldo; }
}
```

Herança (extends)

```
class Animal {}
class Cao extends Animal { // Cao herda Animal
    // Reutiliza/especializa
}
```

Override (Sobrescrita)

```
class Animal { void som() {} }
class Cao extends Animal {
    @Override
    void som() {
        System.out.println("Au!");
    }
}
```

Interface (implements)

```
interface Autentica {
    void login(); // Público e abstrato
}
class User implements Autentica {
    public void login() {} // Obrigatório impl.
}
```

instanceof

```
Object obj = "teste";
if (obj instanceof String) { // Testa o tipo
    String s = (String) obj;
}
```

equals()

```
// Comparação lógica
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof User)) return false;
    User u = (User) o;
    return this.nome.equals(u.nome);
}
```

Composição

```
// Relação forte/dependente
class Humano {
    private final Coracao c = new Coracao(); // Morre junto
}
```

Atributos

```
public class Pessoa {
    String nome; // Instância (objeto)
    int idade = 0; // Valor padrão
    static String especie; // Classe (compartilhado)
}
```

this

```
public class Ponto {
    int x;
    public Ponto(int x) {
        this.x = x; // Atributo da classe
    }
    public void print() {
        this.desenhar(); // Método da classe
    }
    private void desenhar() {}
}
```

Setter

```
public class Pessoa {
    private int idade;
    public void setIdade(int i) {
        if (i >= 0) this.idade = i; // Validação
    }
}
```

Polimorfismo

```
// Objeto assume várias formas
Animal a1 = new Cachorro();
Animal a2 = new Gato();
a1.falar(); // Latido (runtime)
a2.falar(); // Miado (runtime)
```

Overload (Sobrecarga)

```
class Calc {
    int add(int a) { return a + 1; }
    int add(int a, int b) { return a + b; } // Assinatura dif.
}
```

static

```
class MathUtil {
    static final double PI = 3.14; // Constante classe
    static int dobro(int x) { // Método classe
        return x * 2;
    }
}
```

Cast (Classes)

```
Animal a = new Gato(); // Upcast (automático)
Gato g = (Gato) a; // Downcast (explícito)
```

enum

```
// Tipo com valores fixos
enum Nivel { BAIXO, MEDIO, ALTO }
Nivel n = Nivel.MEDIO;
```

Agregação

```
// Relação fraca/independente
class Departamento {
    List<Prof> profs; // Se dep sumir, profs ficam
}
```